

LECTURE 24

Database Authorization

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- Why database security matters: confidentiality, integrity, availability
- Database users, privileges, and discretionary vs. mandatory access control
- Granting and revoking privileges with GRANT and REVOKE
- Roles and role-based access control
- Views as a security mechanism, tying back to Lecture 23
- A consolidated reference, plus SQL injection and least privilege

Authentication vs. Authorization

Database security is the protection of data against unauthorized access, modification, or destruction.

Authentication: who are you?

Proving identity, for example by logging in with a password. Happens first, once per session.

Authorization: what may you do?

Deciding whether this operation, by this user, on this object, is allowed. Enforced on every statement.

The CIA Triad

Security rests on three classic goals, the CIA triad:

1

Confidentiality

Only authorized users can READ the data. A student should not read other students' grades.

2

Integrity

Only authorized users can MODIFY the data, and only in permitted ways. A student cannot edit their own grade.

3

Availability

Authorized users can reach the data when they need it. Controls should not block legitimate access.

Database Users and Privileges

Every user holds privileges: specific permissions on specific database objects.

Privilege	Lets the holder
SELECT	Read rows from a table or view
INSERT	Add new rows
UPDATE	Modify existing rows
DELETE	Remove rows
EXECUTE	Run a stored procedure or function
ALL PRIVILEGES	Shorthand for every applicable privilege

Discretionary vs. Mandatory Access Control

Discretionary (DAC)

The owner of an object, or an administrator, decides who gets which privileges.

This is what GRANT and REVOKE implement, and what almost every relational DBMS uses.

Mandatory (MAC)

Every object and user has a fixed classification. Access follows a system-wide policy.

Mainly government and military systems. This course focuses on DAC.

Granting Privileges

GRANT gives one or more privileges, on one object, to one or more users or roles.

Basic GRANT statements

```
GRANT SELECT ON Staff TO usman;  
GRANT SELECT, INSERT, UPDATE ON Staff TO usman, ayesha;  
GRANT ALL PRIVILEGES ON Branch TO db_admin;
```

By default, the recipient can use the privilege but cannot pass it along.

Column-Level Grants

Column-level grants restrict a privilege to specific columns only.

Finer-grained privileges

```
GRANT UPDATE (position, salary) ON Staff TO hr_clerk;  
GRANT EXECUTE ON ProcessPayroll TO payroll_service;
```

hr_clerk can change position and salary, and nothing else

Useful when a user must update some fields but never others on the same table.

Delegating with WITH GRANT OPTION

WITH GRANT OPTION also lets the recipient grant the same privilege to others.

Delegating the privilege

```
GRANT SELECT ON Staff TO branch_manager WITH GRANT OPTION;
```

GRANT OPTION spreads responsibility, not just access

branch_manager can now run GRANT SELECT ON Staff TO someone_else without the DBA. Privilege chains can grow past what a single audit of top-level GRANTS shows, so use it sparingly.

Revoking Privileges

REVOKE is the mirror image of GRANT, with matching syntax.

Removing privileges

```
REVOKE UPDATE ON Staff FROM usman;  
REVOKE ALL PRIVILEGES ON Branch FROM db_admin;  
REVOKE SELECT ON Staff FROM branch_manager CASCADE;
```

- Revoking a privilege granted WITH GRANT OPTION cascades: anything granted onward because of it is revoked too.
- CASCADE makes that explicit. Where supported, RESTRICT blocks a revoke that would cascade.

Roles and Role-Based Access Control

Granting to dozens of individual users does not scale. RBAC adds one layer: a role is a named bundle of privileges.

1. CREATE ROLE

Define a named role, for example `branch_staff`.



2. GRANT privileges TO the role

Bundle every privilege the role needs, once.



3. GRANT the role TO users

Every assigned user inherits the whole bundle immediately.

RBAC in Practice

Set up the role once

```
CREATE ROLE branch_staff;  
GRANT SELECT, INSERT, UPDATE ON Staff TO branch_staff;  
GRANT SELECT ON Branch TO branch_staff;  
GRANT branch_staff TO usman, ayesha, hamza;
```

Policy changes happen in one place

```
GRANT DELETE ON Staff TO branch_staff;    -- everyone gets it  
REVOKE branch_staff FROM hamza;           -- hamza loses it all
```

Grant to roles, assign users to roles

Never the other way around, unless a privilege is unique to one person.

Authorization Rules

An authorization rule is the DBMS's internal record of which privilege a user or role holds on an object.

grantor	grantee	object	privilege	grantable?
DBA	usman	Staff	SELECT	no
DBA	branch_manager	Staff	SELECT	yes

Checked on every single statement

No matching rule means the statement is rejected outright, before anything executes. Authorization is not a one-time login check.

Restricting Columns with a View

Hide salary from anyone who is not HR, by granting on a restricted view instead.

Column restriction

```
CREATE VIEW StaffPublicInfo AS
  SELECT staffNo, name, position, branchNo
  FROM Staff;
GRANT SELECT ON StaffPublicInfo TO general_staff;
```

Structural, not a matter of discipline

general_staff never gets SELECT on Staff itself, so salary stays invisible to them.

Restricting Rows with a View

Let each branch manager see only their own branch's staff.

Row restriction

```
CREATE VIEW MyBranchStaff AS
  SELECT staffNo, name, position, salary
  FROM Staff
  WHERE branchNo = CURRENT_BRANCH(); -- resolved per session
GRANT SELECT ON MyBranchStaff TO branch_manager;
```

No way around the filter

The manager has no privilege on Staff itself, so querying the base table is not possible.

SQL Authorization Commands

Command	Purpose
GRANT <priv> ON <obj> TO <user role>	Give privileges on an object
GRANT ... WITH GRANT OPTION	Give privileges, plus the right to re-grant
REVOKE <priv> ON <obj> FROM <user role>	Remove previously granted privileges
REVOKE ... CASCADE	Remove, and cascade to anything granted onward
CREATE ROLE <name>	Define a new named role
GRANT <role> TO <user>	Assign a user to a role
REVOKE <role> FROM <user>	Remove a user from a role
CREATE VIEW ... AS SELECT ...	Define a restricted row/column subset

SQL Injection

The most common real-world database attack: untrusted input concatenated directly into SQL.

How the attack works

```
-- Application builds this string from user input:  
"SELECT * FROM Staff WHERE staffNo = '" + userInput + "'"
```



```
-- Attacker submits, as userInput:    ' OR '1'='1'  
-- Statement actually executed:  
SELECT * FROM Staff WHERE staffNo = '' OR '1'='1'
```

Result: every row is returned

The intended filter is bypassed entirely. A chained payload can even run a second statement.

The Fix: Parameterized Queries

The fix is a parameterized query: user input is bound as a parameter and is never parsed as SQL.

Prepared statement

```
PREPARE getStaff AS SELECT * FROM Staff WHERE staffNo = $1;  
EXECUTE getStaff('SG37');
```

- Authorization and parameterized queries are complementary, not substitutes.
- Even a perfectly parameterized app should connect with an account that holds only the privileges it needs.

The Least-Privilege Principle

Grant a user, or an application's own database account, exactly the privileges its job requires, and nothing more.

Example: a reporting dashboard

Its account needs `SELECT` and nothing else. It should never hold `DELETE` on any table.

Why it matters

A bug or a successful injection cannot destroy data the account was never allowed to touch.

Key Takeaways

- Authorization continuously decides who may do what to which data, resting on the CIA triad.
- SQL implements discretionary access control: GRANT assigns privileges, REVOKE removes them, and WITH GRANT OPTION allows re-granting.
- Grant privileges to roles and assign users to roles. One statement changes a whole policy.
- Views double as security: grant on a restricted view instead of the base table to enforce row and column limits.
- Use parameterized queries against SQL injection, and least privilege to limit the damage when a defense fails.